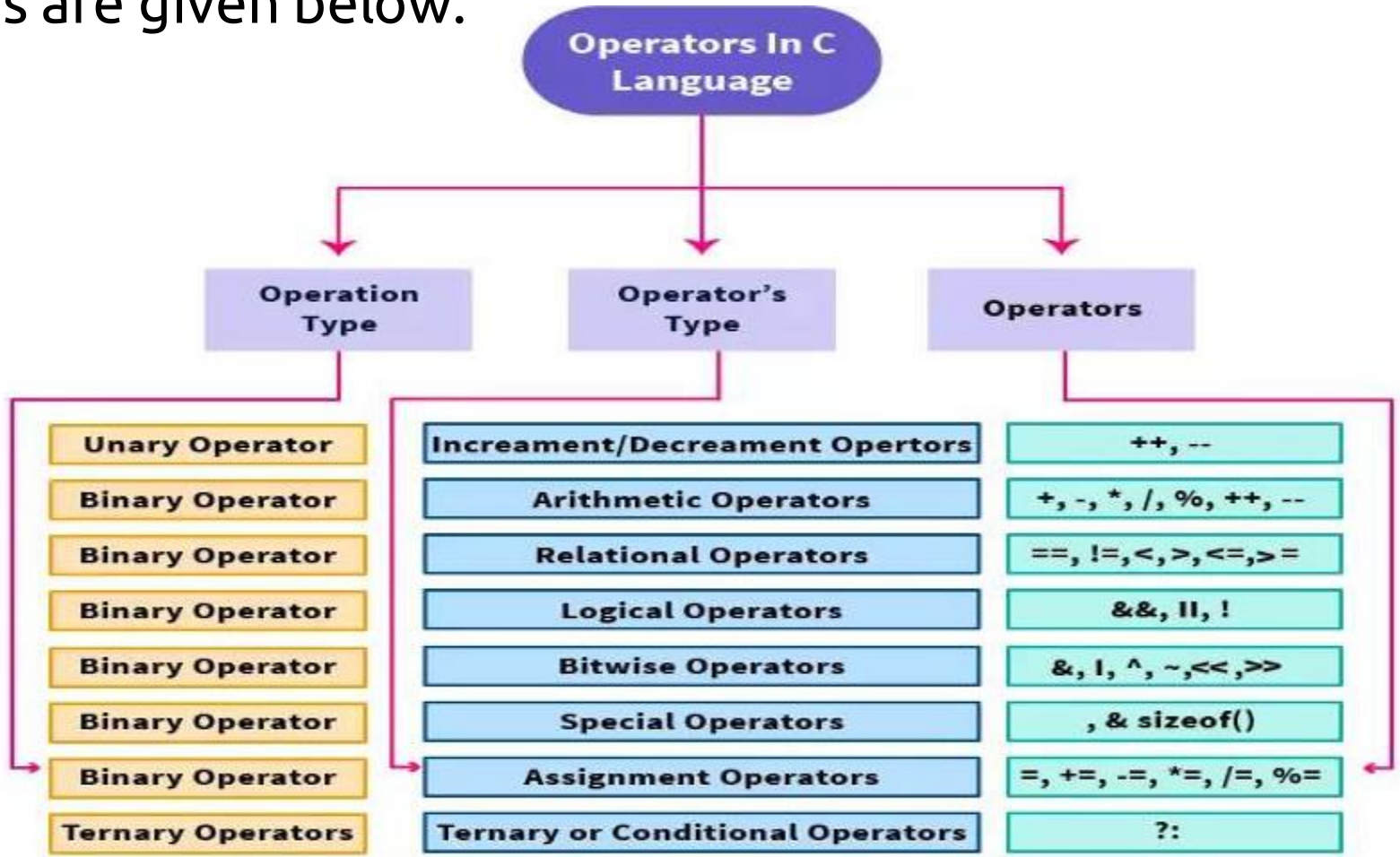
C UNIT-2

Complete Notes

OPERATORS

An operator is a symbol that tells the computer to perform certain mathematical or logical manipulation on data stored in variables. The variables that are operated are termed as operands. C operators can be classified into 8 types. These operators are given below.



Contents of Unit-2

- Operators and Expression – Arithmetic, relational, logical, assignment, increment, decrement, conditional, bitwise, special operators; Type conversion in expressions; Operator precedence and associativity.

- Decision making, Branching and Looping - If statement- Simple if statement, if-else statement, nested if-else statements, else-if ladder; switch statement; ternary operator; continue and break statement; while statement; do-while statement; for statement.

1. ARITHMETIC OPERATORS

Definition: Used to perform basic mathematical operations.

Operator	Meaning	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division (quotient)	a / b
%	Modulus (remainder)	a % b

```
#include <stdio.h>
int main()
{
int a = 10, b = 3;
printf("a + b = %d\n", a + b);
printf("a - b = %d\n", a - b);
printf("a * b = %d\n", a * b);
printf("a / b = %d\n", a / b);
printf("a % b = %d\n", a % b);
return 0;
}
```

Output

```
a + b = 13
a - b = 7
a * b = 30
a / b = 3
a 1 = 1
```

2. RELATIONAL OPERATORS

Definition: Used to compare two values, result is either true (1) or false (0).

Operator	Meaning	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater or equal	a >= b
<=	Less or equal	a <= b

```
#include <stdio.h>
int main()
{
int a = 5, b = 10;
printf("a == b : %d\n", a == b);
printf("a != b : %d\n", a != b);
printf("a > b : %d\n", a > b);
printf("a < b : %d\n", a < b);
printf("a >= b : %d\n", a >= b);
printf("a <= b : %d\n", a <= b);
return 0;
}
```

Output

```
a == b : 0
a != b : 1
a > b : 0
a < b : 1
a >= b : 0
a <= b : 1
```

3. LOGICAL OPERATORS

Used to combine relational expressions (conditions).

Operator	Meaning	Example
&&	Logical AND (true if both are true)	(a > 0 && b > 0)
||	Logical OR (true if one or both are true)	(a > 0 || b > 0)
!	Logical NOT (reverses result)	!(a > b)

```
#include<stdio.h>
int main()
 {
int x = 1, y = 0;
printf("x && y = %d\n", x && y);
printf("x || y = %d\n", x || y);
printf("!x = %d\n", !x);
printf("!y = %d\n", !y);
return 0;
}
```

Output

```
x && y = 0
x || y = 1
!x = 0
!y = 1
```

4. ASSIGNMENT OPERATORS

Used to assign values to variables, can also perform operation + assignment together

Operator	Meaning	Example
=	Simple assignment	a = 5
+=	Add and assign	a += 3 → a = a + 3
-=	Subtract and assign	a -= 2 → a = a - 2
*=	Multiply and assign	a *= 4 → a = a * 4
/=	Divide and assign	a /= 2 → a = a / 2
%=	Modulus and assign	a %= 3 → a = a % 3

```
#include <stdio.h>
 int main()
{
 int a = 10;

 printf("Initial a = %d\n", a);
 a += 5;

printf("After a += 5 : %d\n", a);
 a -= 2;

printf("After a -= 2 : %d\n", a);
 a *= 3;

printf("After a *= 3 : %d\n", a);
 a /= 4;

printf("After a /= 4 : %d\n", a);
 a %= 3;

printf("After a %= 3 : %d\n", a);
 return 0;
 }
```

Output

```
Initial a = 10
After a += 5 : 15
After a -= 2 : 13
After a *= 3 : 39
After a /= 4 : 9
After a %= 3 : 0
```

5. BITWISE OPERATORS

These operators work bit by bit (on binary representation of integers).

Operator	Meaning	Example
&	Bitwise AND	a & b
|	|	Bitwise OR
^	Bitwise XOR (exclusive OR)	a ^ b
~	Bitwise NOT (one's complement)	~a
<<	Left Shift	a << 1 (multiply by 2)
>>	Right Shift	a >> 1 (divide by 2)

```
#include<stdio.h>
int main()
{
int a = 5, b = 3;
printf("a & b = %d\n", a & b);
printf("a | b = %d\n", a | b);
printf("a ^ b = %d\n", a ^ b);
printf("~a = %d\n", ~a);
printf("a << 1 = %d\n", a << 1);
printf("a >> 1 = %d\n", a >> 1);
return 0;
}
```

Output

```
a & b = 1
a | b = 7
a ^ b = 6
~a = -6
a << 1 = 10
a >> 1 = 2
```

6. INCREMENT AND DECREMENTS OPERATORS

Increment operator is used to increment the current value of variable by adding integer 1 and can be applied to only variables denoted by ++.

Two types of increment operator Pre-Increment Operator - is used to increment the value of variable before using in the expression

Example: a=5; b = ++a;

The value of b will be 6 because 'a' is incremented first and then assigned to 'b'.value of 'a' is 6

Post-Increment Operator - is used to increment the value of variable after executing expression in which post increment is used.

Example: a=5; b = a++;

The value of b will be 5 because 'b' is assigned first and then value of 'a' is incremented to 6

```
#include<stdio.h> void
main()
{
int a,b,x=100,y=100;
a = x++;
b = ++y;
printf("Value of a : %d\n",a);
printf("Value of b : %d\n",b);
}
```

Output

```
Value of a : 100
Value of b : 101
```

Decrement operator is used to decrease the current value of variable by subtracting integer 1 and can be applied to only variables and is denoted by --.

Pre-decrement operator :It is used to decrement the value of variable before using in the expression. Example: a=10; b=--a;

Value of 'b' will be 9 and 'a' will also be 9.

Post-decrement Operator : In this the old value is first used in a expression and then old value will be decrement by 1.

Example: a=10; b=a--;

Value of 'b' will be 10 and 'a' will be 9

```
#include<stdio.h>
void main()
{
int a, b, x = 100, y = 100;
a = x--;
b = --y; s
printf("Value of a : %d\n",a);
printf("Value of b : %d\n",b);
}
```

Output

```
Value of a : 100
Value of b : 99
```

7. CONDITIONAL OPERATOR

The? : Operator The conditional operator ? : can be used to replace if...else statements. It has the following general form:

Exp1 ? Exp2 : Exp3;

where Exp1, Exp2, and Exp3 are expressions.

The value of a ? expression is determined like this: Exp1 is evaluated.

If it is true, then Exp2 is evaluated and becomes the value of the entire ? expression.

If Exp1 is false, then Exp3 is evaluated and its value becomes the value of the expression.

This is also called as Ternary Operator because it uses three operands.

```
#include<stdio.h>
 int main()
{
int num;
printf("Enter the Number : ");
scanf("%d",&num);
(num%2==0)?printf("Even"):printf("Odd");
}
```

Output

```
Enter the Number : 5
Odd
```

7. SPECIAL OPERATORS

There are few other important operators in C.

They are:

1. sizeof( )

Returns the size of an variable.

Example:

```
#include <stdio.h>
int main()
{
printf("Size of int: %zu bytes\n", sizeof(int));
printf("Size of float: %zu bytes\n", sizeof(float));
printf("Size of char: %zu byte\n", sizeof(char));
return 0;
}
```

2. &

Returns the address of an variable.

Example:

```
#include <stdio.h>
  int main()
  {
    int num = 10;
    printf("Address of num: %p\n", &num);
    return 0;

  }
```

3. *

Pointer to a variable. It is variable which saves the address of another variable

```
#include <stdio.h>
int main()
{
  int* pc, c;
    c = 22;
  printf("Address of c: %p\n", &c);
  printf("Value of c: %d\n\n", c);  // 22
    pc = &c;
  printf("Address of pointer pc: %p\n", pc);
  printf("Content of pointer pc: %d\n\n", *pc); // 22
  return 0;
}
```

4. , Comma Operator

separate the declaration of multiple variables

PRECEDENCE

Operator precedence determines the grouping of terms in an expression. This affects how an expression is evaluated. Certain operators have higher precedence (priority) than others.

For example x = 7 + 3 * 2; Here x is assigned 13, not 20 because operator * has higher precedence than + so it first get multiplied with 3*2 and then adds into 7.

Here operators with the highest precedence appear at the top of the table and those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

ASSOCIATIVITY

The associativity determines the direction in which an expression is evaluated.

Example:

1 == 2 != 3 ,the associativity is from left to right

a =b , associativity is from right to left

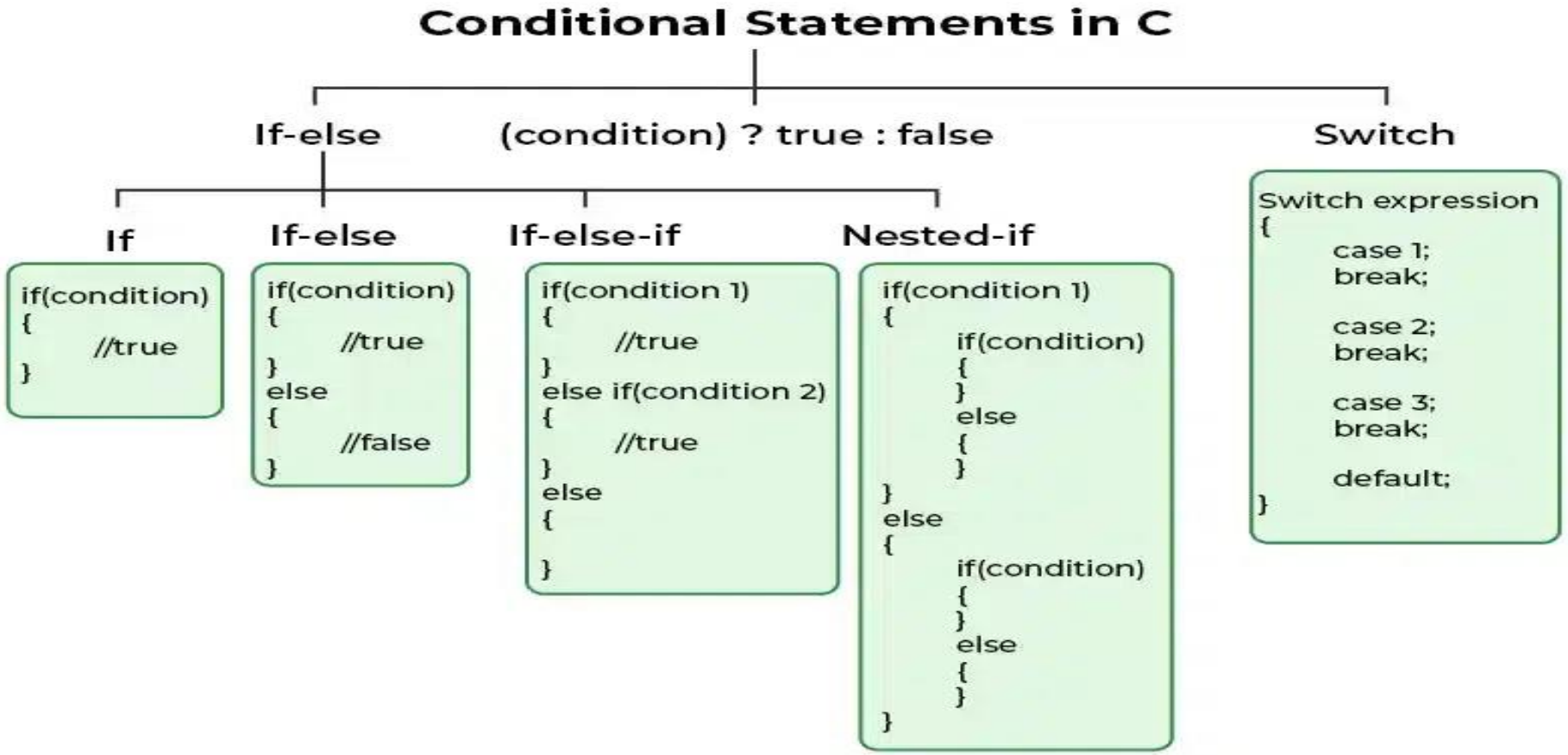
PRECEDENCE & ASSOCIATIVITY

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type) * & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= |=	Right to left
Comma	,	Left to right

DECISION MAKING, BRANCHING AND LOOPING

Decision making structures allows the programmer to specify one or more conditions to be evaluated or tested by the program, along wit h a statement or statements to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false.

TYPES OF CONDITIONAL STATEMENTS IN C



1. IF IN C

The <u>if statement</u> is the simplest decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e if a certain condition is true then a block of statements is executed otherwise not.

A condition is any expression that evaluates to either a true or false (or values convertible to true or false).

```
#include <stdio.h>
int main( )
{
        int i = 10;
        // If statement
          if (i < 18)
                {
                        printf("Eligible for vote");
                }
}
```

2. IF-ELSE IN C

The **if** statement alone tells us that if a condition is true, it will execute a block of statements and if the condition is false, it won't. But what if we want to do something else when the condition is false? Here comes the C **else** statement. We can use the **else** statement with the **if** statement to execute a block of code when the condition is false. The if-else statement consists of two blocks, one for false expression and one for true expression.

```c
#include <stdio.h>

int main() {
    int i = 10;

    if (i > 18) {
        printf("Eligible for vote");
    }
    else {
        printf("Not Eligible for vote");
    }
    return 0;
}
```

3. NESTED IF-ELSE IN C

A nested if in C is an if statement that is the target of another if statement. Nested if statements mean an if statement inside another if statement. Yes, C allow us to nested if statements within if statements, i.e, we can place an if statement inside another if statement.

```c
#include <stdio.h>

int main(){
    int i = 10;

    if (i == 10) {
        if (i < 18)
            printf("Still not eligible for vote");
        else
            printf("Eligible for vote\n");
    }
    else {
        if (i == 20) {
            if (i < 22)
                printf("i is smaller than 22 too\n");
            else
                printf("i is greater than 25");
        }
    }

    return 0;
}
```

4. IF-ELSE-IF LADDER IN C

The if else if statements are used when the user has to decide among multiple options. The C if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the C else-if ladder is bypassed. If none of the conditions is true, then the final else statement will be executed. if-else-if ladder is similar to the switch statement.

```c
#include <stdio.h>

int main() {
    int i = 20;

    // If else ladder with three conditions
    if (i == 10)
        printf("Not Eligible");
    else if (i == 15)
        printf("wait for three years");
    else if (i == 20)
        printf("You can vote");
    else
        printf("Not a valid age");

    return 0;
}
```

5. SWITCH STATEMENT IN C

The switch case statement is an alternative to the if else if ladder that can be used to execute the conditional code based on the value of the variable specified in the switch statement. The switch block consists of cases to be executed based on the value of the switch variable.

```c
int main() {

    // variable to be used in switch statement
    int var = 18;

    // declaring switch cases
    switch (var) {
    case 15:
        printf("You are a kid");
        break;
    case 18:
        printf("Eligible for vote");
        break;
    default:
        printf("Default Case is executed");
        break;
    }

    return 0;
}
```

6. CONDITIONAL OPERATOR IN C

The conditional operator is used to add conditional code in our program. It is similar to the if-else statement. It is also known as the ternary operator as it works on three operands.

```c
#include <stdio.h>

int main() {
    int var;
    int flag = 0;

    // using conditional operator to assign the value to var
    // according to the value of flag
    var = flag == 0 ? 25 : -25;
    printf("Value of var when flag is 0: %d\n", var);

    return 0;
}
```

7. JUMP STATEMENTS IN C

These statements are used in C for the unconditional flow of control throughout the functions in a program. They support four types of jump statements:

A) break

B) continue

C) goto

D) return

A) BREAK

This loop control statement is used to terminate the loop. As soon as the break statement is encountered from within a loop, the loop iterations stop there, and control returns from the loop immediately to the first statement after the loop.

```c
#include <stdio.h>

int main() {
    int arr[] = { 1, 2, 3, 4, 5, 6 };
    int key = 3;
    int size = 6;

    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            printf("Element found at position: %d",
                   (i + 1));
            break;
        }
    }

    return 0;
}
```

B) CONTINUE

This loop control statement is just like the break statement. The *continue* statement is opposite to that of the break *statement*, instead of terminating the loop, it forces to execute the next iteration of the loop.

As the name suggests the continue statement forces the loop to continue or execute the next iteration. When the continue statement is executed in the loop, the code inside the loop following the continue statement will be skipped and the next iteration of the loop will begin.

```c
#include <stdio.h>

int main() {
    for (int i = 1; i <= 10; i++) {

        // If i is equals to 6,
        // continue to next iteration
        // without printing
        if (i == 6)
            continue;
        else
            printf("%d ", i);
    }

    return 0;
}
```

C) GOTO

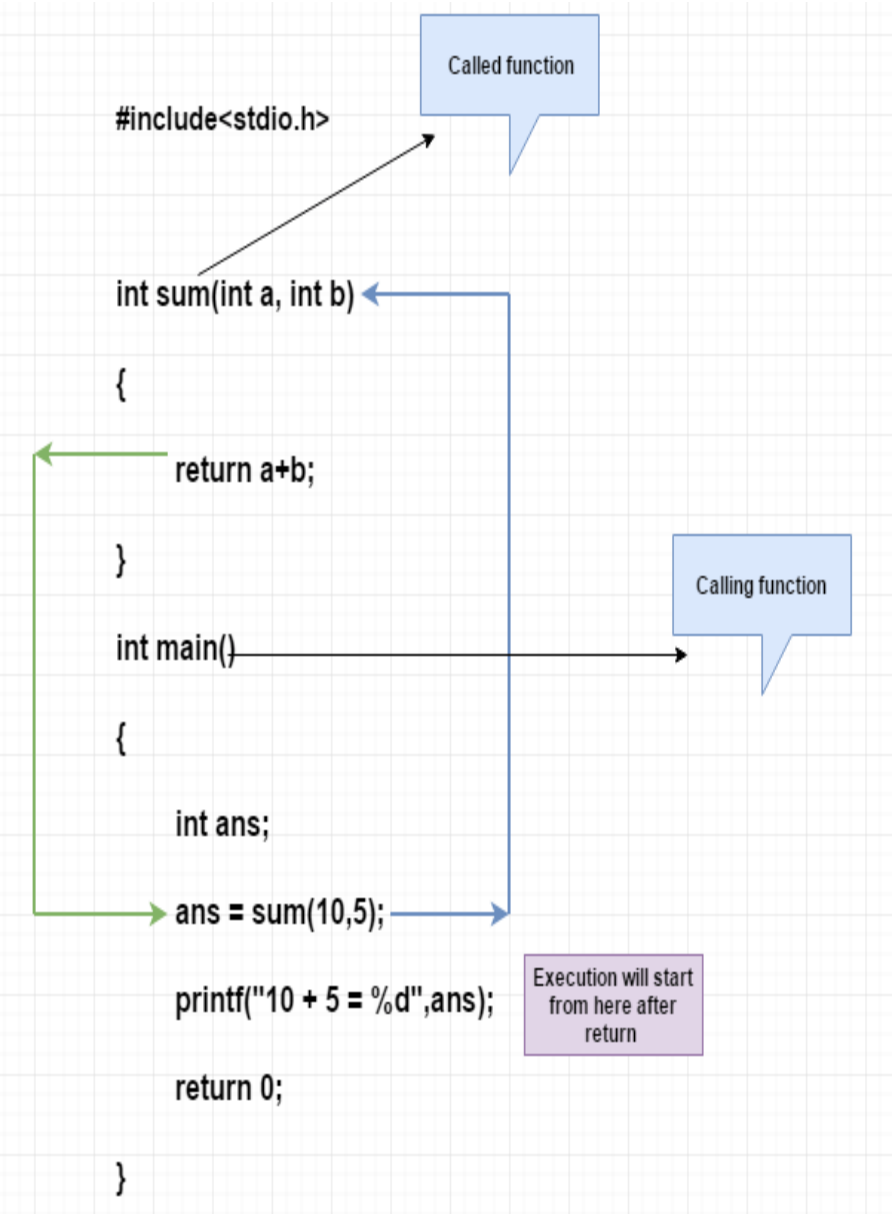
The goto statement in C also referred to as the unconditional jump statement can be used to jump from one point to another within a function.

```c
#include <stdio.h>

int main() {
    int n = 1;
label:
    printf("%d ", n);
    n++;
    if (n <= 10)
        goto label;
    return 0;
}
```

D) RETURN

The return in C returns the flow of the execution to the function from where it is called. This statement does not mandatorily need any conditional statements. As soon as the statement is executed, the flow of the program stops immediately and returns the control from where it was called. The return statement may or may not return anything for a void function, but for a non-void function, a return value must be returned.



TURN OFF PHONE
PUT THE PHONE DOWN..
PUT THE PHONE DOWN..NOWWW
don't give up.
don't give up.
don't give up.
don't give up.
don't give up.
don't give up.
don't give up.
don't give up.
don't give up.
don't give up.
HURRY UP.. RUN NOWWW..
STUDY NOW..
Not much time left..
Study now..noww..
Fighting..
I believe youuu..